

Lesson 2

Java Basics

In this lesson you will:

- use the numeric type `int` to count the number of students
- use the Java collection class `java.util.ArrayList` to store many students
- understand default constructors
- learn how to use the J2SE API documentation to understand how to use `java.util.ArrayList`
- restrict the `java.util.ArrayList` collection to contain only `Student` objects
- create a `TestSuite` to test more than one class
- learn about packages and the `import` statement
- understand how to define and use class constants
- use date and calendar classes from the system library
- learn the various types of comments that Java allows
- generate API documentation for your code with `javadoc`

CourseSession

CourseSession



Schools have courses, such as Math 101 and Engl 200, that are taught every semester. Basic course information, such as the department, the number, the number of credits, and the description of the course, generally remains the same from semester to semester.



A course session represents a specific occurrence of a course. A course session stores the dates it will be held and the person teaching

it, among other things. It also must retain an enrollment, or list of students, for the course.

You will define a `CourseSession` class that captures both the basic course information and the enrollment in the session. As long as you only need to work with the `CourseSession` objects for a single semester, no two `CourseSessions` should need to refer to the same course. Once two `CourseSession` objects must exist for the same course, having the basic course information stored in both `CourseSession` objects is redundant. For now, multiple sessions is not a consideration; later you will clean up the design to support multiple sessions for a single course.

Create `CourseSessionTest.java`. Within it, write a test named `testCreate`. Like the `testCreate` method in `StudentTest`, this test method will demonstrate how you create `CourseSession` objects. A creation test is always a good place to get a handle on what an object looks like just after it's been created.

CourseSession

```
public class CourseSessionTest extends junit.framework.TestCase {
    public void testCreate() {
        CourseSession session = new CourseSession("ENGL", "101");
        assertEquals("ENGL", session.getDepartment());
        assertEquals("101", session.getNumber());
    }
}
```

The test shows that a `CourseSession` can be created with a course department and number. The test also ensures that the department and number are stored correctly in the `CourseSession` object.

To get the test to pass, code `CourseSession` like this:

```
class CourseSession {
    private String department;
    private String number;

    CourseSession(String department, String number) {
        this.department = department;
        this.number = number;
    }

    String getDepartment() {
        return department;
    }

    String getNumber() {
        return number;
    }
}
```

So far you've created a `Student` class that stores student data and a `CourseSession` class that stores course data. Both classes provide “getter” methods to allow other objects to retrieve the data.

However, data classes such as `Student` and `CourseSession` aren't terribly interesting. If all there was to object-oriented development was storing data and retrieving it, systems wouldn't be very useful. They also wouldn't be object-oriented. Remember that object-oriented systems are about modeling behavior. That behavior is effected by sending messages to objects to get them to do something—*not* to ask them for data.

But, you've got to start somewhere! Plus, you wouldn't be able to write assertions in your test if you weren't able to ask objects what they look like.

Enrolling Students



Courses don't earn any revenue for the school unless students enroll in them. Much of the student information system will require you to be able to work with more than one student at a time. You will want to store groups, or collections, of students and later execute operations against the students in these collections.

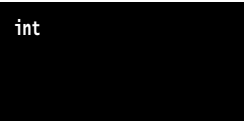
`CourseSession` will need to store a new attribute—a collection of `Student` objects. You will want to bolster your `CourseSession` creation test so that it says something about this new attribute. If you have just created a new course session, you haven't yet enrolled any students in it. What can you assert against an empty course session?

Modify `testCreate` so that it contains the **bolded** assertion:

```
public void testCreate() {
    CourseSession session = new CourseSession("ENGL", "101");
    assertEquals("ENGL", session.getDepartment());
    assertEquals("101", session.getNumber());
    assertEquals(0, session.getNumberOfStudents());
}
```

int

The new assertion verifies that the number of students enrolled in a brand-new session should be 0 (zero). The symbol 0 is a *numeric literal* that represents the integer zero. Specifically, it is an integer literal, known in Java as an `int`.



int

Add the method `getNumberOfStudents` to `CourseSession` as follows:

```
class CourseSession {
    ...
    int getNumberOfStudents() {
        return 0;
    }
}
```

(The ellipses represent the instance variables, constructor code, and getter methods that you have already coded.) The return type of `getNumberOfStudents` is specified as `int`. The value returned from a method must match the return type, and in this method it does—`getNumberOfStudents` returns an `int`. The `int` type allows variables to be created that store integer values from $-2,147,483,648$ to $2,147,483,647$.

Numbers in Java are not objects like `String` literals are. You cannot send messages to numbers, although numbers can be passed as parameters along with messages just like `Strings` can. Basic arithmetic support in Java is provided syntactically; for many other operations, support is provided by system libraries. You will learn about similar non-object types later in Agile Java. As a whole, these non-object types are known as *primitive types*.

You have proved that a new `CourseSession` object is initialized properly, but you haven't shown that the class can enroll students properly. Create a second test method `testEnrollStudents` that enrolls two students. For each, create a new `Student` object, enroll the student, and ensure that the `CourseSession` object reports the correct number of students.

```
public class CourseSessionTest extends junit.framework.TestCase {
    public void testCreate() {
        ...
    }

    public void testEnrollStudents() {
        CourseSession session = new CourseSession("ENGL", "101");

        Student student1 = new Student("Cain DiVoe");
        session.enroll(student1);
        assertEquals(1, session.getNumberOfStudents());

        Student student2 = new Student("CoraLee DeV Vaughn");
        session.enroll(student2);
        assertEquals(2, session.getNumberOfStudents());
    }
}
```

How do you know that you need a method named `enroll`, and that it should take a `Student` object as a parameter? Part of what you are doing in a test method is designing the *public interface* into a class—how developers

will interact with the class. Your goal is to design the class such that the needs of developers who want use it are met in as simple a fashion as possible.

The simplest way of getting the second assertion (that the number of students is two) to pass would be to return 2 from the `getNumberOfStudents` method. However, that would break the first assertion. So you must somehow track the number of students inside of `CourseSession`. To do this, you will again introduce a field. Any time you know you need information to be stored, you will likely use fields to represent object *state*. Change the `CourseSession` class to look like this:

```
class CourseSession {  
    ...  
    private int numberOfStudents = 0;  
    ...  
    int getNumberOfStudents() {  
        return numberOfStudents;  
    }  
  
    void enroll(Student student) {  
        numberOfStudents = numberOfStudents + 1;  
    }  
}
```

int

The field to track the student count is named `numberOfStudents`, it is private per good practice, and it is of the type `int`. It is also assigned an *initial value* of 0. When a `CourseSession` object is instantiated, *field initializers* such as this initialization of `numberOfStudents` are executed. Field initializers are executed prior to the invocation of code in the constructor.

The method `getNumberOfStudents` now returns the field `numberOfStudents`, instead of the `int` literal 0.

Each time the `enroll` method is called, you should increment the number of students by one. The single line of code in the `enroll` method accomplishes this:

```
numberOfStudents = numberOfStudents + 1;
```

The `+` sign and many other mathematical operators are available for working with variables of the `int` type (as well as other numeric types to be discussed later). The expression on the right hand side of the `=` sign takes whatever value is currently stored in `numberOfStudents` and adds 1 to it. Since the `numberOfStudents` field appears to the left of the `=` sign, the resultant value of the right-hand side expression is assigned back into it. Bumping up a variable's value by one, as in this example, is a common operation known as

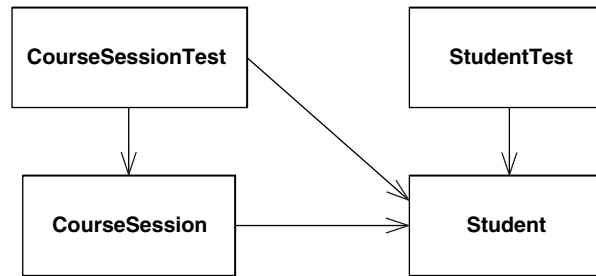


Figure 2.1 *CourseSession and Student Class Diagram*

incrementing the variable. There are other ways to increment a variable that will be discussed later.

It may seem odd that `numberOfStudents` appears on both sides of the assignment operator. Remember that the Java VM always executes the right hand side of an assignment statement first. It calculates a result using the expression to the right, and assigns this result to the variable on the left.

Note that the `enroll` method has a return type `void`, meaning that it returns nothing to the message sender.

The class diagram in Figure 2.1 shows the existing structure of your system so far.

Conceptually, a course session should be able to hold several students. In reality—in the code—the course session holds references to no student objects. It only holds a count of students. Later, when you modify the `CourseSession` class to actually store `Student` references, the UML diagram will be modified to show that there is a one-to-many relationship between `CourseSession` and `Student`.

`CourseSession` depends on `Student`, since the `enroll` method can take a `Student` object as a parameter. In other words, you would not be able to compile the `CourseSession` class if the `Student` class did not exist.

Figure 2.1 will be the last class diagram where I show every test class. Since you are doing test-driven development, future diagrams will imply the existence of a test class for each production class, unless otherwise noted.

Initialization

Initialization

In the last section, you introduced the `numberOfStudents` field and initialized it to 0. This initialization is technically not required—`int` fields are initialized to 0 by default. Explicitly initializing a field in this manner, while unnecessary, is useful to help explain the intent of the code.

For now, you have two ways to initialize fields: You can initialize at the field level or you can initialize in a constructor. You could have initialized `numberOfStudents` in the `CourseSession` constructor:

```
class CourseSession {
    private String department;
    private String number;
    private int numberOfStudents;

    CourseSession(String department, String number) {
        this.department = department;
        this.number = number;
        numberOfStudents = 0;
    }
    ...
}
```

There is no hard-and-fast rule about where to initialize. I prefer initializing at the field level when possible—having the initialization and declaration in one place makes it easier to follow the code. Also, as you will learn later in this lesson, you can have more than one constructor; initializing at the field level saves you from duplicate initialization code in each constructor.

You will encounter situations where you cannot initialize code at the field level. Initializing in the constructor may be your only alternative.

Default Constructors

Default Constructors

You may have noted that neither of the test classes, `StudentTest` and `CourseSessionTest`, contains a constructor. Often you will not need to explicitly initialize anything, so the Java compiler does not require you to define a constructor. If you do not define any constructors in a class,¹ Java provides a default, no-argument constructor. For `StudentTest`, as an example, it is as if you had coded an empty constructor:

```
class StudentTest extends junit.framework.TestCase {
    StudentTest() {
    }
    ...
}
```

The use of default constructors also implies that Java views constructors as essential elements to a class. A constructor is required in order for Java to initialize a class, even if the constructor contains no additional initialization

¹The Java compiler does allow you to define multiple constructors in a single class; this will be discussed later.

code. If you don't supply a constructor, the Java compiler puts it there for you.

Suites

In the last section, you introduced a second test class, `CourseSessionTest`. Going forward, you could decide to run JUnit against either `CourseSessionTest` or `StudentTest`, depending on which corresponding production class you changed. Unfortunately, it is very possible for you to make a change in the `Student` class that breaks a test in `CourseSessionTest`, yet all the tests in `StudentTest` run successfully.

You could run the tests in `CourseSessionTest` and then run the tests in `StudentTest`, either by restarting JUnit each time or by retyping the test class name in JUnit or even by keeping multiple JUnit windows open. None of these solutions is scalable: As you add more classes, things will quickly become unmanageable.

Instead, JUnit allows you to build *suites*, or collections of tests. Suites can also contain other suites. You can run a suite in the JUnit test runner just like any test class.

Create a new class called `AllTests` with the following code:

```
public class AllTests {
    public static junit.framework.TestSuite suite() {
        junit.framework.TestSuite suite =
            new junit.framework.TestSuite();
        suite.addTestSuite(StudentTest.class);
        suite.addTestSuite(CourseSessionTest.class);
        return suite;
    }
}
```

If you start JUnit with the class name `AllTests`, it will run all of the tests in both `CourseSessionTest` and `StudentTest` combined. From now on, run `AllTests` instead of any of the individual class tests.

The job of the `suite` method in `AllTests` is to build the suite of classes to be tested and return it. An object of the type `junit.framework.TestSuite` manages this suite. You add tests to the suite by sending it the message `addTestSuite`. The parameter you send with the message is a *class literal*. A class literal is comprised of the name of the class followed by `.class`. It uniquely identifies the class, and lets the class definition itself be treated much like any other object.

Each time you add a new test class, you will need to remember to add it to the suite built by `AllTests`. This is an error-prone technique, as it's easy to for-

get to update the suite. In Lesson 12, you'll build a better solution: a tool that generates and executes the suite for you.

The code in `AllTests` also introduces the concept of *static methods*, something you will learn about in Lesson 4. For now, understand that you must declare the suite method as static in order for JUnit to recognize it.

The SDK and `java.util.ArrayList`

Your `CourseSession` class tracks the count of students just fine. However, you and I both know that it's just maintaining a counter and not holding onto the students that are being enrolled. For the `testEnrollStudents` method to be complete, it will need to prove that the `CourseSession` object is retaining the actual student objects.

One possible solution is to ask the `CourseSession` for a list of all the enrolled students, then check the list to ensure that it contains the expected students. Modify your test to include the lines shown below in bold.

The SDK and
`java.util.ArrayList`

```
public void testEnrollStudents() {
    CourseSession session = new CourseSession("ENGL", "101");

    Student student1 = new Student("Cain DiVoe");
    session.enroll(student1);
    assertEquals(1, session.getNumberOfStudents());
    java.util.ArrayList<Student> allStudents = session.getAllStudents();
    assertEquals(1, allStudents.size());
    assertEquals(student1, allStudents.get(0));

    Student student2 = new Student("CoraLee DeV Vaughn");
    session.enroll(student2);
    assertEquals(2, session.getNumberOfStudents());
    assertEquals(2, allStudents.size());
    assertEquals(student1, allStudents.get(0));
    assertEquals(student2, allStudents.get(1));
}
```

The first line of newly added test code:

```
java.util.ArrayList<Student> allStudents = session.getAllStudents();
```

says that there will be a method on `CourseSession` named `getAllStudents`. The method will have to return an object of the type `java.util.ArrayList<Student>`, since that's the type of the variable to which the result of `getAllStudents` is to be assigned. A type appearing in this form—a class name followed by a parameter type within angle brackets (< and >) is known as a *parameterized type*. In the example, the parameter `Student` of

`java.util.ArrayList` indicates that the `java.util.ArrayList` is *bound* such that it can only contain `Student` objects.

The class `java.util.ArrayList` is one of thousands available as part of the Java SDK class library. You should have downloaded and installed the SDK documentation so that it is available on your machine. You can also browse the documentation online at Sun's Java site. My preference is to have the documentation available locally for speed and accessibility reasons.

Pull up the documentation and navigate to the Java 2 Platform API Specification. You should see something like Figure 2.2.

The Java API documentation will be your best friend, unless you're doing *pair programming*,² in which case it will be your second-best friend. It is divided into three frames. The upper left frame lists all the *packages* that are available in the library. A package is a group of related classes.

The lower left frame defaults to showing all the classes in the library. Once a package is selected, the lower left frame shows only the classes contained within that package. The frame to the right that represents the remainder of the page shows detailed information on the currently selected package or class.

Scroll the package frame until you see the package named `java.util` and select it. The lower left pane should now show a list of *interfaces*, classes, and

The SDK and
`java.util.ArrayList`

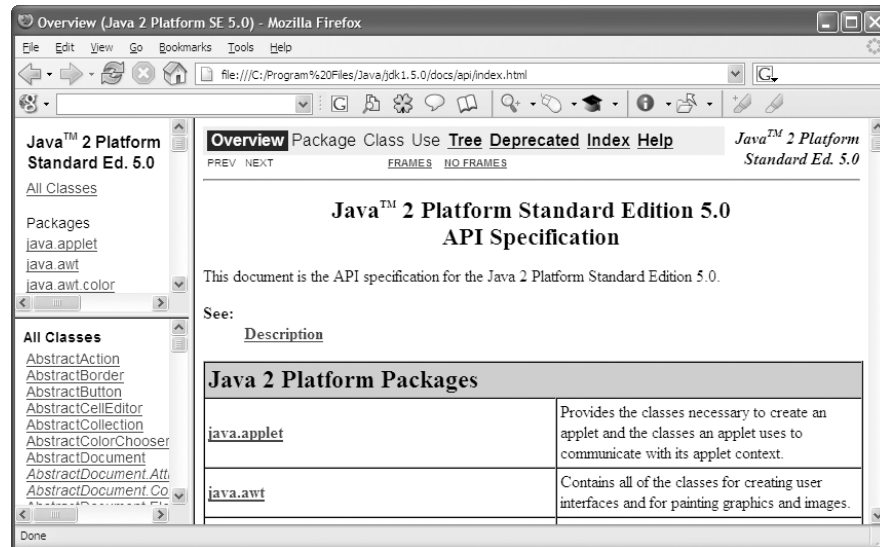


Figure 2.2 Java 2 Platform API Specification

²A technique whereby members of a development team work in dynamic pairs of developers who jointly construct code.

exceptions. I will explain interfaces and exceptions later. For now, scroll down until you see the class named `ArrayList` and select it.

The package `java.util` contains several utility classes that you will use often in the course of Java development. The bulk of the classes in the package support something called the Collections Framework. The Collections Framework is a consistent sublibrary of code used to support standard data structures such as lists, linked lists, sets, and hash tables. You will use collections over and over again in Java to work with groupings of related objects.

The main pane (to the right) shows all of the detailed information on the class `java.util.ArrayList`. Scroll down until you see the Method Summary. The Method Summary shows the methods implemented in the `java.util.ArrayList` class. Take a few minutes to read through the methods available. Each of the method names can be clicked on for detailed information regarding the method.

You will use three of the `java.util.ArrayList` methods as part of the current exercise: `add`, `get`, and `size`. One of them, `size`, is already referenced by the test. The following line of code asserts that there is one object in the `java.util.ArrayList` object.

```
assertEquals(1, allStudents.size());
```

The next line of new code asserts that the first element in `allStudents` is equal to the student that was enrolled.

```
assertEquals(student, allStudents.get(0));
```

According to the API documentation, the `get` method returns the element at an arbitrary position in the list. This position is passed to the `get` method as an index. Indexes are zero-based, so `get(0)` returns the first element in the list.

Adding Objects

Adding Objects

The `add` method is documented in the Java SDK API as taking an `Object` as parameter. If you click on the parameter `Object` in the API docs, you will see that it is actually `java.lang.Object`.

You might have heard that Java is a “pure” object-oriented language, where “everything is an object.”³ The class `java.lang.Object` is the mother of all classes defined in the Java system class library, as well as of any class that

³A significant part of the language is not object-oriented. This will be discussed shortly.

you define, including `Student` and `StudentTest`. Every class inherits from `java.lang.Object`, either directly or indirectly. `StudentTest` inherits from `junit.framework.TestCase`, and `junit.framework.TestCase` inherits from `java.lang.Object`.

This inheritance from `java.lang.Object` is important: You'll learn about several core language features that depend on `java.lang.Object`. For now, you need to understand that `String` inherits from `java.lang.Object`, as does `Student`, as does any other class you define. This inheritance means that `String` objects and `Student` objects are also `java.lang.Object` objects. The benefit is that `String` and `Student` objects can be passed as parameters to any method that takes a `java.lang.Object` as a parameter. As mentioned, the `add` method takes an instance of `java.lang.Object` as a parameter.

Even though you can pass any type of object to a `java.util.ArrayList` via its `add` method, you don't usually want to do that. In the `CourseSession` object, you know that you only want to be able to enroll `Students`. Hence the parameterized type declaration. One of the benefits to restricting the `java.util.ArrayList` via a parameterized type is that it protects other types of objects from inadvertently being added to the list.

Suppose someone is allowed to add, say, a `String` object to the list of students. Subsequently, your code asks for the list of students using `getAllStudents` and retrieves the `String` object from the list using the `get` method. When you attempt to assign this object to a `Student` reference, the Java VM will choke, generating an error condition. Java is a *strongly typed* language, and it will not allow you to assign a `String` to a `Student` reference.

The following changes to `CourseSession` (appearing in bold) will make the test pass:

```
class CourseSession {
    ...
    private java.util.ArrayList<Student> students =
        new java.util.ArrayList<Student>();
    ...
    void enroll(Student student) {
        numberOfStudents = numberOfStudents + 1;
        students.add(student);
    }

    java.util.ArrayList<Student> getAllStudents() {
        return students;
    }
}
```

A new field, `students`, is used to store the list of all students. It is initialized to an empty `java.util.ArrayList` object that is bound to contain only `Student`

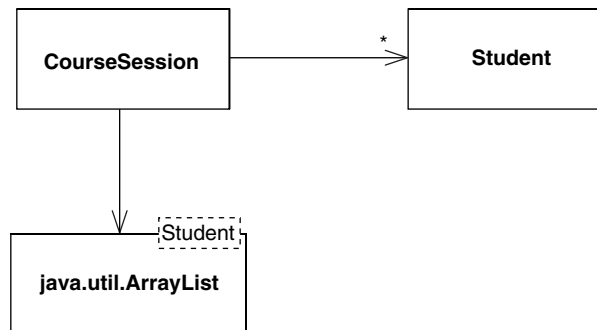


Figure 2.3 Class Diagram with Parameterized Type

objects.⁴ The `enroll` method adds the student to this list, and the `getAllStudents` method simply returns the list.

Figure 2.3 shows how `CourseSession` is now dependent upon the parameterized type `java.util.ArrayList<Student>`. Also, the fact that `CourseSession` is dependent upon zero to many students is shown by the `*` at the end of the association between `CourseSession` and `Student`.

The class diagram in Figure 2.3 is not normal—it really shows the same information twice, in a different fashion. The parameterized type declaration suggests that a single `CourseSession` has a relationship to a number of `Student` objects stored in an `ArrayList`. The association from `CourseSession` to `Student` similarly shows a one-to-many relationship.

Incremental
Refactoring

Incremental Refactoring

Since `java.util.ArrayList` provides a `size` method, you can just ask the students `ArrayList` object for its size instead of tracking `numberOfStudents` separately.

```
int getNumberOfStudents() {
    return students.size();
}
```

Make this small refactoring, then recompile and rerun the test. Having the test gives you the confidence to change the code with impunity—if the change doesn't work, you simply undo it and try something different.

⁴Due to width restrictions, I've broken the `students` field declaration into two lines. You can type this as a single line if you choose.

Since you no longer return `numberOfStudents` from `getNumberOfStudents`, you can stop incrementing it in the `enroll` method. This also means you can delete the `numberOfStudents` field entirely.

While you could scan the `CourseSession` class for uses of `numberOfStudents`, removing each individually, you can also use the compiler as a tool. Delete the field declaration, then recompile. The compiler will generate errors on all the locations where the field is still referenced. You can use this information to navigate directly to the code you need to delete.

The final version of the class should look like the following code:

```
class CourseSession {
    private String department;
    private String number;
    private java.util.ArrayList<Student> students =
        new java.util.ArrayList<Student>();

    CourseSession(String department, String number) {
        this.department = department;
        this.number = number;
    }

    String getDepartment() {
        return department;
    }

    String getNumber() {
        return number;
    }

    int getNumberOfStudents() {
        return students.size();
    }

    void enroll(Student student) {
        students.add(student);
    }

    java.util.ArrayList<Student> getAllStudents() {
        return students;
    }
}
```

Objects in Memory

Objects in Memory

In `testEnrollStudents`, you send the `getAllStudents` message to `session` and store the result in a `java.util.ArrayList` reference that is bound to the `Student` class—it can only contain `Student` objects. Later in the test, after enrolling the second

student, `allStudents` now contains both students—you don't need to ask the session object for it again.

```
public void testEnrollStudents() {
    CourseSession session = new CourseSession("ENGL", "101");

    Student student1 = new Student("Cain DiVoe");
    session.enroll(student1);
    assertEquals(1, session.getNumberOfStudents());
    java.util.ArrayList<Student> allStudents = session.getAllStudents();
    assertEquals(1, allStudents.size());
    assertEquals(student1, allStudents.get(0));

    Student student2 = new Student("Coralee DeVaughn");
    session.enroll(student2);
    assertEquals(2, session.getNumberOfStudents());
    assertEquals(2, allStudents.size());
    assertEquals(student1, allStudents.get(0));
    assertEquals(student2, allStudents.get(1));
}
```

Objects in
Memory

The reason is illustrated in Figure 2.4. The `CourseSession` object holds on to the students field as an attribute. This means that the students field is available throughout the lifetime of the Course-Session object. Each time the `getNumberOfStudents` message is sent to the session, a reference to the very same students field is returned. This reference is a memory address, meaning that

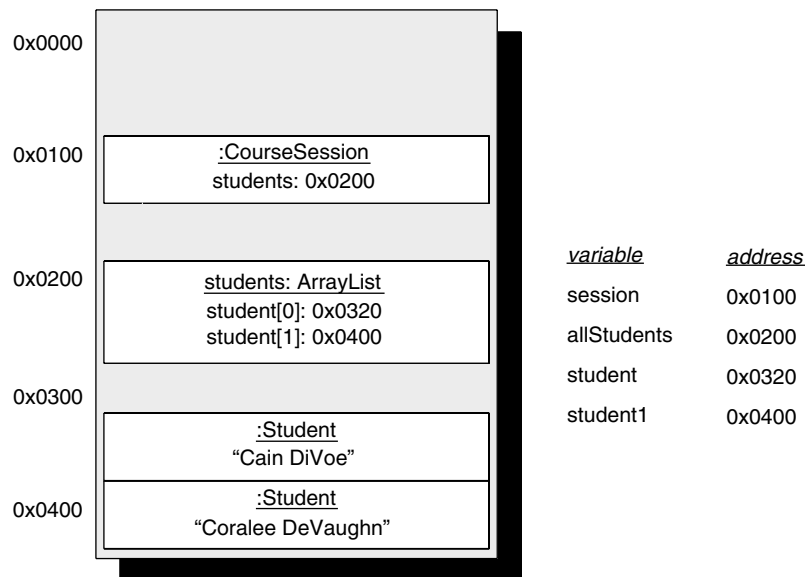


Figure 2.4 Memory Diagram

any code using the reference ends up at the same memory location at which the `students` field is stored.

Packages and the `import` Statement

Up until now, you have been using the *fully qualified* name of the class `java.util.ArrayList`. The fully qualified name of a class includes its package name (`java.util` in this example) followed by the class name (`ArrayList`).

Packages provide a way for developers to group related classes. They can serve several needs: First, grouping classes into packages can make it considerably easier on developers, saving them from having to navigate dozens, hundreds, or even thousands of classes at a time. Second, classes can be grouped into packages for distribution purposes, perhaps to allow easier reuse of modules or subsystems of code.

Third, packages provide *namespaces* in Java. Suppose you have built a class called `Student` and you purchase a third-party API to handle billing students for tuition. If the third-party software contains a class also named `Student`, any references to `Student` will be ambiguous. Packages provide a way to give a class a more unique name, minimizing the potential for class name conflicts. Your class might have a fully qualified name of `com.mycompany.studentinfosystem.Student`, and the third-party API might use the name `com.thirdpartyco.expensivepackage.Student`.

I will usually refer to the Java system classes without the package name unless the package is not clear from the class name. For example, I will use `ArrayList` in place of `java.util.ArrayList`, and `Object` instead of `java.lang.Object`.

Typing `java.util.ArrayList` throughout code can begin to get tedious, and it clutters the code as well. Java provides a keyword—`import`—that allows identification of fully qualified class names and/or packages at the source file level. Use of `import` statements allows you to specify simple class names throughout the remainder of the source file.

Update `CourseSessionTest` to include `import` statements as the very first lines in the source file. You can now shorten the phrase `extends junit.framework.TestCase` to `extends TestCase`. You can change the reference defined as `java.util.ArrayList<Student>` to `ArrayList<Student>`.

```
import junit.framework.TestCase;
import java.util.ArrayList;

public class CourseSessionTest extends TestCase {
    ...
}
```



```

public void testEnrollStudents() {
    CourseSession session = new CourseSession("ENGL", "101");

    Student student1 = new Student("Cain DiVoe");
    session.enroll(student1);
    assertEquals(1, session.getNumberOfStudents());
    ArrayList<Student> allStudents = session.getAllStudents();
    ...
}
}

```

(Make sure your tests still run! I'll keep reminding you for a while.)

Update `AllTests`, `StudentTest`, and `CourseSession` to use `import` statements. Your code will look so much cleaner!

The java.lang Package

The `String` class is also part of the system class library. It belongs to the package named `java.lang`. So why haven't you had to either use its fully qualified class name (`java.lang.String`) or provide an `import` statement?

The Java library contains classes so fundamental to Java programming that they are needed in many, if not all, classes. The classes `String` and `Object` are two such classes. The ubiquitous nature of these classes is such that the designers of Java wanted to save you from the nuisance of having to specify an `import` statement for them everywhere.

If you refer to the `String` class, then the statement:

```
import java.lang.String;
```

is implicit in each and every Java source file.

When you learn about inheritance (Lesson 6), you will find that every class implicitly extends from the class `java.lang.Object`. In other words, if a class declaration does not contain an `extends` keyword, it is as if you coded:

```
class ClassName extends java.lang.Object
```

This is another shortcut provided by the Java compiler.

**The Default
Package and
the package
Statement**

The Default Package and the package Statement

None of the classes you have built—`AllTests`, `StudentTest`, `Student`, `CourseSessionTest`, and `CourseSession`—specify a package. This means that they go into something called the *default package*. The default package is fine for

sample or academic programs. But for any real software development you do, all of your classes should belong to some package other than the default package. In fact, if you place classes in the default package, you will not be able to reference these classes from other packages.

If you are working in a modern IDE, moving a class into a package can be as easy as dragging the class name onto a package name. If you are not working in an IDE, setting up packages and understanding related classpath issues can be somewhat complex. Even if you are working in an IDE, it is important to understand the relationship between packages and the underlying file system directory structure.

Let's try moving your classes into a package named `studentinfo`. Note that package names are by convention comprised of lowercase letters. While you should still avoid abbreviations, package names can get unwieldy fairly quickly. The judicious use of abbreviations might help keep the package name from being unmanageable.

You should also not start your package names with `java` or `javax`, both of which should be used exclusively by Sun.

If your IDE supports moving classes into packages as a simple operation, go ahead and take advantage of it. However, make sure you understand how the classes relate to a package structure, as this will be important when you construct deployable libraries. Consider copying the source files into a different directory structure and following the exercise anyway.

From the directory in which your class files are located, create a new subdirectory called `studentinfo`. Case is important—ensure that you name this subdirectory using all lowercase letters. If your source files are currently located in `c:\source`, then you should now have a directory named `c:\source\studentinfo`. Starting with a Unix directory named `/usr/src`, you should have a directory named `/usr/src/studentinfo`.

First carefully delete all of the class files that have been generated. Class files, remember, end with a `.class` extension. Don't lose all of your hard work—back up your directory if you are unsure about this step. Under Windows, the command:

```
del *.class
```

will suffice. Under Unix, the equivalent command is:

```
rm *.class
```

After deleting the class files, move the five source files (`AllTests.java`, `StudentTest.java`, `Student.java`, `CourseSessionTest.java`, and `CourseSession.java`) into the `studentinfo` directory. You need to store classes in a directory structure that corresponds to the package name.

Edit each of the five Java source files. Add a package statement to each that identifies the class as belonging to the `studentinfo` package. The package statement must be the very first statement within a Java class file.

```
package studentinfo;
```

```
class Student {  
...
```

You should be able to compile the classes from within the `studentinfo` subdirectory.

However, two things will have to change when you run JUnit against `AllTests`. First, the full name of the class will have to be passed into the `TestRunner`, otherwise JUnit will not find it. Second, if you are in the `studentinfo` directory, the `TestRunner` will not be able to find `studentinfo.AllTests`. You must either go back to the parent directory or, better yet, alter the classpath to explicitly point to the parent directory. The following command shows the altered classpath and the fully qualified test class name.

```
java -cp c:\source;c:\junit3.8.1\junit.jar junit.awtui.TestRunner studentinfo.AllTests
```

In fact, if you explicitly point to `c:\source` in the classpath, you can be in any directory when you execute the `java` command. You will also want to use a similar classpath in `javac` compilations:

```
javac -classpath c:\source;c:\junit3.8.1\junit.jar studentinfo\*.java
```

Another way to look at this: The classpath specifies the starting, or “root,” directory in which to find any classes that you will use. The class files must be located in the subdirectory of one of the roots that corresponds to the package name. For example, if the classpath is `c:\source` and you want to include the class `com.mycompany.Bogus`, then the file `Bogus.class` must appear in `c:\source\com\mycompany`.

A package-naming standard is something that your development team will want to agree upon. Most companies name their packages starting with the reverse of their web domain name. For example, software produced by a company named Minderbinder Enterprises might use package names starting with `com.minderbinder`.

The `setUp`
Method

The `setUp` Method

The test code in `CourseSessionTest` needs a bit of cleanup work. Note that both tests, `testCreate` and `testEnrollStudents`, instantiate a new `CourseSession` object and store a reference to it in a local variable named `session`.

JUnit provides you with a means to eliminate this duplication—a `setUp` method. If you provide code in this `setUp` method, JUnit will execute that code prior to executing each and every test method. You should put common test initialization code in this method.

```
public class CourseSessionTest extends TestCase {
    private CourseSession session;

    public void setUp() {
        session = new CourseSession("ENGL", "101");
    }

    public void testCreate() {
        assertEquals("ENGL", session.getDepartment());
        assertEquals("101", session.getNumber());
        assertEquals(0, session.getNumberOfStudents());
    }

    public void testEnrollStudents() {
        Student student1 = new Student("Cain DiVoe");
        session.enroll(student1);
        ...
    }
}
```

The `setUp` Method

Caution!

It's easy to make the mistake of coding `setUp` so that it declares `session` as a local variable:

```
public void setUp() {
    CourseSession session =
        new CourseSession("ENGL", "101");
}
```

It is legal to define a local variable with the same name as an instance variable, but it means that the *instance* variable `session` will not get properly initialized. The result is an error known as a `NullPointerException`, which you'll learn about in Lesson 4.

In `CourseSessionTest`, you add the instance variable `session` and assign to it a new `CourseSession` instance created in the `setUp` method. The test methods `testCreate` and `testEnrollStudents` no longer need this initialization line. Both test methods get their own separate `CourseSession` instance.

Even though you could create a constructor and supply common initialization code in it, doing so is considered bad practice. The preferred idiom for test initialization in JUnit is to use the `setUp` method.

More Refactoring

The method `testEnrollStudents` is a bit longer than necessary. It contains a few too many assertions that track the number of students. Overall, the method is somewhat difficult to follow.

Instead of exposing the entire list of students to *client* code (i.e., other code that works with `CourseSession` objects), you can instead ask the `CourseSession` to return the `Student` object at a specific index. You currently have no need for the entire list of students as a whole, so you can eliminate the `getAllStudents` method. This also means that you no longer have to test the size of the `ArrayList` returned by `getAllStudents`. The test method can be simplified to this:

```
public void testEnrollStudents() {
    Student student1 = new Student("Cain DiVoe");
    session.enroll(student1);
    assertEquals(1, session.getNumberOfStudents());
    assertEquals(student1, session.get(0));

    Student student2 = new Student("Coralee DeVaughn");
    session.enroll(student2);
    assertEquals(2, session.getNumberOfStudents());
    assertEquals(student1, session.get(0));
    assertEquals(student2, session.get(1));
}
```

Add the `get` method to `CourseSession` and remove the method `getAllStudents`. This refactoring shows how you can move common code from the test class directly into the production class in order to remove duplication.

```
class CourseSession {
    ...
    ArrayList<Student> getAllStudents() {
        return students;
    }

    Student get(int index) {
        return students.get(index);
    }
}
```

Another significant benefit of this refactoring is that you have hidden a detail of `CourseSession` that was unnecessarily exposed. You have *encapsulated* the students collection, only selectively allowing the `get`, `add`, and `size` operations to be performed on it.

This encapsulation provides two significant benefits: First, you currently store the list of in an `ArrayList`. An `ArrayList` is one specific kind of data

structure with certain use and performance characteristics. If you expose this list directly to client code, their code now depends upon the fact that the students are available as an `ArrayList`. This dependency means that later you cannot easily change the representation of how students are stored. Second, exposing the entire collection means that other classes can manipulate that collection—add new `Student` objects, remove `Student` objects, and so on—without your `CourseSession` class being aware of these changes. The integrity of your `CourseSession` object could be violated!

Class Constants

It is never a good idea to embed literals directly in code, as mentioned earlier. Declaring a local variable as `final`, which prevents other code from reassigning its value, is a good idea. The `final` keyword tells other developers that “I don’t intend for you to be able to change the value of this variable.”

Class Constants

Idiosyncracies: The Final Keyword

You can mark local object and primitive variables as `final`:

```
public void testCreate() {  
    final String firstStudentName =  
        "Jane Doe";  
    final Student firstStudent =  
        new Student(firstStudentName);  
}
```

You can also mark arguments as `final`:

```
Student(final String name) {  
    this.name = name;  
}
```

Doing so can provide you with an additional level of protection from other code in the method that assigns a different value to the local variable or argument. Many developers insist on doing so, and it’s not a bad practice.

I choose not to, but I recommend that you try it and see how it helps you. My general reason to mark fields as `final` is for readability, not protection. As a rule, you should never assign values to arguments. And you should rarely reassign values to local object reference variables. I follow both of these rules and don’t feel the need to demonstrate that fact by marking the field as `final`. Instead, I use `final` to emphasize the fact that a local declaration is a literal constant and not just an initialized variable. Your mileage may vary.

Frequently you will need more than one class to use the same literal. In fact, if you are doing test-driven development properly, any time you create a literal in code, it is there by virtue of some assertion in a test method against that same literal. A test might say, “assert that these conditions produce an error and that the error message is such-and-such a message.” Once this test is written, you then write the production code that produces such-and-such an error message under the right conditions. Now you have code duplication—“such-and-such an error message” appears in both the test and the production code.

While a few duplicated strings here and there may seem innocuous, overlooking the need to refactor out this duplication may prove costly in the future. One possibility is related to the growth of your software. Initially you may only deploy your system to local customers. Later, as your software achieves more success, the business may decide that it needs to deploy the software in another country. You will then need to *internationalize* your software, by deploying it with support for other languages and with consideration for other cultures.

Many software developers have encountered this problem. The software they worked on for months or years has hundreds or thousands of literal Strings strewn throughout the code. It is a major effort to retrofit internationalization support into this software.

As a craftsman of Java code, it is your job to stay vigilant and ensure that duplication is eliminated as soon as you recognize it.⁵



Replace common literals with class constants.

A class constant is a field that is declared with the `static` and `final` keywords. As a reminder, the `final` keyword indicates that the field reference cannot be changed to point to a different value. The `static` keyword means that the field is available for use without having to first construct an instance of the class in which it is declared. It also means that there is one and only one field in memory instead of one field for each object created. The following example declares a class constant:

```
class ChessBoard {  
    static final int SQUARES_PER_SIDE = 8;  
}
```

⁵For the problem of duplicated strings, a more robust solution involving *resource bundles* is more appropriate. See Additional Lesson III for a brief discussion of resource bundles.

By convention, class constants appear in uppercase letters. When all letters are uppercase, camel notation is not possible, so the standard is to use underscores to separate words.

You can refer to the class constant by first specifying the class name, followed by the dot (.) operator, followed by the name of the constant.

```
int numberOfSquares =  
    ChessBoard.SQUARES_PER_SIDE * ChessBoard.SQUARES_PER_SIDE;
```

You will use class constants already defined in the Java class library in the next section on Dates. Shortly thereafter you will define your own class constant in the `CourseSession` code.

Overloaded Constructors

Dates

If you browse through the J2SE API documentation for the package `java.util`, you will see several classes that are related to times and dates, including `Calendar`, `GregorianCalendar`, `Date`, `TimeZone`, and `SimpleTimeZone`. The `Date` class provides a simple timestamp mechanism. The other, related classes work together with `Date` to provide exhaustive support for internationalized dates and for working with components of a timestamp.

Initial versions of Java shipped with the `Date` class as the sole provider of support for dates and times. The `Date` class was designed to provide most of the functionality needed. It is a simple implementation: Internally, a date is represented by the number of milliseconds (thousandths of a second) since January 1, 1970 GMT at 00:00:00 (known as the “epoch”).

Overloaded Constructors

The `Date` class provides a handful of constructors. It is possible and often desirable to provide a developer with more than one way to construct new objects of a class type. You will learn how to create multiple constructors for your class in this lesson.

In the `Date` class, three of the constructors allow you to specify time parts (year, month, day, hour, minute, or second) in order to build a `Date` object for a specific date and time. A fourth constructor allows you to construct a date from an input `String`. A fifth constructor allows you to construct a date using the number milliseconds since the epoch. A final constructor, one that

takes no parameters, constructs a timestamp that represents “now,” the time at which the `Date` object was created.

There are also many methods that allow getting and setting of fields on the `Date`, such as `setHour`, `setMinutes`, `getDate`, and `getSeconds`.

The `Date` class was not designed to provide support for internationalized dates, however, so the designers of Java introduced the `Calendar` classes in J2SE 1.1. The intent was for the `Calendar` classes to supplement the `Date` class. The `Calendar` class provides the ability to work with dates by their constituent parts. This meant that the constructors and getters/setters in the `Date` class were no longer needed as of J2SE 1.1. The cleanest approach would have been for Sun to simply remove the offending constructors and methods.

However, if Sun were to have changed the `Date` class, large numbers of existing applications would have had to have been recoded, recompiled, retested, and redeployed. Sun chose to avoid this unhappy circumstance by instead *deprecating* the constructors and methods in `Date`. This means that the methods and constructors are still available for use, but the API developers are warning you that they will remove the deprecated code from the next major release of Java. If you browse the `Date` class in the API documentation, you will see that Sun has clearly marked the deprecated methods and constructors.

In this exercise, you will actually use the deprecated methods. You’ll see that their use generates warnings from the compiler. Warnings are for the most part bad—they indicate that you’re doing something in code that you probably should not be. There’s always a better way that does not generate a warning. As the exercise progresses, you will eliminate the warnings using an improved solution.



In the student information system, course sessions need to have a start and end date, marking the first and last days of class. You could provide both start and end dates to a `CourseSession` constructor, but you have been told that sessions are always 16 weeks (15 weeks of class, with a one-week break after week 7). With this information, you decide to design the `CourseSession` class so that users of the class need only supply the start date—your class will calculate the end date.

Here’s the test to be added to `CourseSessionTest`.

```
public void testCourseDates() {
    int year = 103;
    int month = 0;
    int date = 6;
    Date startDate = new Date(year, month, date);

    CourseSession session =
        new CourseSession("ABCD", "200", startDate);
}
```

```

    year = 103;
    month = 3;
    date = 25;
    Date sixteenWeeksOut = new Date(year, month, date);
    assertEquals(sixteenWeeksOut, session.getEndDate());
}

```

You will need an `import` statement at the top of `CourseSessionTest`:

```
import java.util.Date;
```

This code uses one of the deprecated constructors for `Date`. Also of note are the odd-looking parameter values being passed to the `Date` constructor. Year 103? Month 0?

The API documentation, which should be your first reference for understanding a system library class, explains what to pass in for the parameters. Specifically, documentation for the `Date` constructor says that the first parameter represents “the year minus 1900,” the second parameter represents “the month between 0 and 11,” and the third parameter represents “the day of the month between 1-31.” So `new Date(103, 0, 6)` would create a `Date` representation for January 6, 2003. Lovely.

Since the start date is so critical to the definition of a course session, you want the constructor to require the start date. The test method constructs a new `CourseSession` object, passing in a newly constructed `Date` object in addition to the department and course number. You will want to change the instantiation of `CourseSession` in the `setUp` method to use this modified constructor. But as an interim, incremental approach, you can instead supply an additional, *overloaded* constructor.

The test finally asserts that the session end date, returned by `getEndDate`, is April 25, 2003.

You will need to make the following changes to `CourseSession` in order to get the test to pass:

- Add `import` statements for `java.util.Date`, `java.util.Calendar`, and `java.util.GregorianCalendar`;
- add a `getEndDate` method that calculates and returns the appropriate session end date; and
- add a new constructor that takes a starting date as a parameter.

The corresponding production code:

```

package studentinfo;

import java.util.ArrayList;
import java.util.Date;

```

```

import java.util.Calendar;
import java.util.GregorianCalendar;

class CourseSession {
    private String department;
    private String number;
    private ArrayList<Student> students = new ArrayList<Student>();
    private Date startDate;

    CourseSession(String department, String number) {
        this.department = department;
        this.number = number;
    }

    CourseSession(String department, String number, Date startDate) {
        this.department = department;
        this.number = number;
        this.startDate = startDate;
    }
    ...
    Date getEndDate() {
        GregorianCalendar calendar = new GregorianCalendar();
        calendar.setTime(startDate);
        int numberOfDays = 16 * 7 - 3;
        calendar.add(Calendar.DAY_OF_YEAR, numberOfDays);
        Date endDate = calendar.getTime();
        return endDate;
    }
}

```

Overloaded Constructors

As I describe what `getEndDate` does, try to follow along in the J2SE API documentation for the classes `GregorianCalendar` and `Calendar`.

In `getEndDate`, you first construct a `GregorianCalendar` object. You then use the `setTime`⁶ method to store the object representing the session start date in the calendar. Next, you create the local variable `numberOfDays` to represent the number of days to be added to the start date in order to come up with the end date. The appropriate number is calculated by multiplying 16 weeks by 7 days per week, then subtracting 3 days (since the last day of the session is on the Friday of the 16th week).

The next line:

```
calendar.add(Calendar.DAY_OF_YEAR, numberOfDays);
```

sends the `add` message to the calendar object. The `add` method in `GregorianCalendar` takes a field and an amount. You will have to look at the J2SE API documentation for `Calendar`—in addition to the documentation for `GregorianCalendar`—to fully understand how to use the `add` method. `GregorianCalendar`

⁶Don't use this as a good example of method naming!

Calendar is a *subclass* of Calendar, which means that the way it works is tied tightly to Calendar. The field that represents the first parameter tells the calendar object what you are adding *to*. In this case, you want to add a number to the day of year. The Calendar class defines DAY_OF_YEAR as well as several other class constants that represent date parts, such as YEAR.

The calendar now contains a date that represents the end date of the course session. You extract this date from the calendar using the `getTime` method and finally return the end date of the course session as the result of the method.

You might wonder if the `getEndDate` method will work, then, when the start date is close to the end of the year. If that can possibly happen, it's time to write a test for it. However, the student information system you are writing is for a university that has been around for 200 years. No semester has ever begun in one year and ended in the next, and it will never happen. In short, you're not going to need to worry about it . . . yet.

The second constructor will be short-lived, but it served the purpose of allowing you to quickly get your new test to pass. You now want to remove the older constructor, since it doesn't initialize the session start date. To do so, you will need to modify the `setUp` method and `testCourseDates`. You should also amend the creation test to verify that the start date is getting stored properly.

Overloaded Constructors

```
package studentinfo;

import junit.framework.TestCase;
import java.util.ArrayList;
import java.util.Date;

public class CourseSessionTest extends TestCase {
    private CourseSession session;
    private Date startDate;

    public void setUp() {
        int year = 103;
        int month = 0;
        int date = 6;
        startDate = new Date(year, month, date);
        session = new CourseSession("ENGL", "101", startDate);
    }

    public void testCreate() {
        assertEquals("ENGL", session.getDepartment());
        assertEquals("101", session.getNumber());
        assertEquals(0, session.getNumberOfStudents());
        assertEquals(startDate, session.getStartDate());
    }
    ...
}
```

```

public void testCourseDates() {
    int year = 103;
    int month = 3;
    int date = 25;
    Date sixteenWeeksOut = new Date(year, month, date);
    assertEquals(sixteenWeeksOut, session.getEndDate());
}
}

```

You can now remove the older constructor from `CourseSession`. You'll also need to add the `getStartDate` method to `CourseSession`:

```

class CourseSession {
    ...
    Date getStartDate() {
        return startDate;
    }
}

```

Deprecation Warnings

Deprecation Warnings

The above code will compile and pass the tests, but you will see a warning or note in the compilation output. If you are using an IDE, you might not see the warnings. Find out how to turn on the warnings—you don't want to hide them.



Eliminate all warnings.

Ignoring a compiler warning is like ignoring a sore tooth—you'll pay for it sooner or later, and it may end up costing you dearly to ignore it for too long.⁷

Note: `CourseSessionTest.java` uses or overrides a deprecated API.

Note: Recompile with `-Xlint:deprecation` for details.

If you are compiling from the command line, you should do just what the message says: Enter the compilation command again, but modify it to include the compilation switch `-Xlint:deprecation` option.

```
javac -classpath c:\junit3.8.1\junit.jar -Xlint:deprecation *.java
```

The new compile output should look something like this:

```

CourseSessionTest.java:15: warning: Date(int,int,int) in java.util.Date has been deprecated
    startDate = new Date(year, month, date);
                      ^

```

⁷He writes, as he sits here with a corrupt molar . . .

```
CourseSessionTest.java:43: warning: Date(int,int,int) in java.util.Date has been deprecated
    Date sixteenWeeksOut = new Date(year, month, date);
                          ^
```

2 warnings

If you are running in an IDE, it will have a setting somewhere to allow you to turn warnings on or off. You might already have seen similar deprecation messages, since warnings are usually “on” by default. In any case, the warnings should be disturbing to your craftsperson sensibilities. May they nag at you incessantly. You will soon learn a different way to construct dates that does not result in the warnings.

The test should run just fine. But you have lots of little ugliness in the code that can be cleaned up.

Refactoring

Refactoring

A quick improvement is to remove the unnecessary local variable, `endDate`, that appears at the end of the method `getEndDate` in `CourseSession`. Declaring the temporary variable aided in your understanding of the `Calendar` class:

```
Date endDate = calendar.getTime();
return endDate;
```

A more concise form is to simply return the `Date` object returned by the call to `getTime`.

```
return calendar.getTime();
```

Import Refactorings

The `CourseSession` class now must import four different classes from the package `java.util`:

```
import java.util.ArrayList;
import java.util.Date;
import java.util.GregorianCalendar;
import java.util.Calendar;
```

This is still reasonable, but you can see that the list could quickly get out of hand as more system library classes are used. It is also a form of duplication—the package name is duplicated in each `import` statement. Remember that your primary refactoring job is to eliminate as much duplication as possible.

Java provides a shortcut version of the `import` statement to declare that you are importing all classes from a specified package:

```
import java.util.*;
```

This form of `import` is known as a *package import*. The asterisk (*) acts as a wildcard character.

After making this change, you can use any additional classes from the `java.util` package without having to modify or add to the `import` statements. Note that there is no runtime penalty for using either this form or the singular class form of the `import` statement. An `import` statement merely declares that classes *might* be used in the class file. An `import` statement does not guarantee that any classes from a package are used in the class file.

There is no consensus about which form is more proper. Most shops use the * form of the `import` statement either all the time or when the number of `import` statements begins to get unwieldy. Some shops insist that all classes must be explicitly named in the `import` statements, which makes it easier to determine the package a class comes from. Modern Java IDEs can enforce the `import` convention your shop decides on and even switch back and forth between the various forms. An IDE makes the choice of which form to use less significant.

It is possible to import a class or package but not use the class or any classes from the package within your source code. The Java compiler will not warn you about these unnecessary `import` statements. Most modern IDEs have optimizing facilities that will help you remove them.

Refactoring

Improving Understanding with a Factory Method

The `getEndDate` method in `CourseSession` is the most complex method you've written yet. It's about as long as you want your methods to get. Most of the methods you write should be between one and half a dozen lines. Some methods might be between half a dozen and a dozen or so lines. If your methods are regularly this length or even longer, you should work on refactoring them. The primary goal is to ensure that methods can be rapidly understood and maintained.

If your methods are short enough, it will be easy to provide a meaningful, concise name for them. If you are finding it difficult to name a method, consider breaking it up into smaller methods, each of which does only one succinctly nameable thing.

Another bit of duplication and lack of clarity that you should be unhappy with lies in the test method. For the time being, you are using the deprecated `Date` constructors, a technique that is considerably simpler than using the

Calendar class. However, the code is somewhat confusing because the year has to be specified as relative to 1900, and the months have been numbered from 0 through 11 instead of 1 through 12.

In `CourseSessionTest`, code a new method called `createDate` that takes more sensible inputs.

```
Date createDate(int year, int month, int date) {
    return new Date(year - 1900, month - 1, date);
}
```

This will allow you to create dates using a 4-digit year and a number from 1 through 12 for the month.

You can now refactor `setUp` and `testCourseDates` to use this utility method. With the introduction of the utility method, defining the local variables `year`, `month`, and `date` adds little to the understanding of the code, since the *factory method*⁸ `createDate` encapsulates some of the confusion. You can eliminate the local variables and embed them directly as parameters in the message send to `createDate`:

Refactoring

```
public void setUp() {
    startDate = createDate(2003, 1, 6);
    session = new CourseSession("ENGL", "101", startDate);
}
...
public void testCourseDates() {
    Date sixteenWeeksOut = createDate(2003, 4, 25);
    assertEquals(sixteenWeeksOut, session.getEndDate());
}
```

Some of you may be shaking your head at this point. You have done a lot of work, such as introducing local variables, and then you have undone much of that same work shortly thereafter.

Part of crafting software is understanding that code is a very malleable form. The best thing you can do is get into the mindset that you are a sculptor of code, shaping and molding it into a better form at all times. Once in a while, you'll add a flourish to make something in the code stand out. Later, you may find that the modification is really sticking out like a sore thumb, asking for someone to soothe it with a better solution. You will learn to recognize these trouble spots in your code ("code smells," as Martin Fowler calls them.)⁹

⁸A method responsible for creating and returning objects. Another, perhaps more concise term, is "creation method."

⁹[Wiki2004].

You will also learn that it is cheaper to fix code problems now rather than wait until the code is too intertwined with the rest of the system. It doesn't take long!



Keep your code clean at all times!

Creating Dates with Calendar

You still receive deprecation warnings each time you compile. Bad, bad. You should get rid of the warnings before someone complains. The benefit of having moved the date creation into a separate method, `createDate`, is that now you will only have to make a change in one place in your code, instead of two, to eliminate the warnings.

Instead of creating a `Date` object using the deprecated constructor, you will use the `GregorianCalendar` class. You can build a date or timestamp from its constituent parts by using the `set` method defined in `Calendar`. The API documentation for `Calendar` lists the various date parts that you can set. The `createDate` method builds a date by supplying a year, a month, and a day of the month.

Creating Dates
with Calendar

```
Date createDate(int year, int month, int date) {
    GregorianCalendar calendar = new GregorianCalendar();
    calendar.clear();
    calendar.set(Calendar.YEAR, year);
    calendar.set(Calendar.MONTH, month - 1);
    calendar.set(Calendar.DAY_OF_MONTH, date);
    return calendar.getTime();
}
```

The `GregorianCalendar` is a bit more sensible than `Date` in that a year is a year. If the real year is 2005, you can pass 2005 to the calendar object instead of 105.

You will need to modify the `import` statements in `CourseSessionTest` in order to compile and test these changes. The simplest way is to import everything from the `java.util` package:

```
import java.util.*;
```

Compile and test. Congratulations—no more embarrassing deprecation warnings!

Comments

One part of the `getEndDate` method that could use clarification is the calculation for the number of days to add to the session start date.

```
int numberOfDays = 16 * 7 - 3;
```

To another developer that has to maintain this method, the mathematical expression is not immediately obvious. While the maintainer can probably figure it out in a few minutes, it's far less expensive for you as the original developer to explain what you were thinking.

Java allows you to add free-form explanatory text within the source file in the form of *comments*. The compiler ignores and discards comments encountered when it reads the source file. It is up to you to decide where and when comments are appropriate.

You can add a *single-line comment* to the `numberOfDays` calculation. A single-line comment begins with two forward slashes (`//`) and continues to the end of the current source line. Anything from the first slash to the end of the line is ignored by the compiler.

```
int numberOfDays = 16 * 7 - 3; // weeks * days per week - 3 days
```

You can place single-line comments on a separate line:

```
// weeks * days per week - 3 days
int numberOfDays = 16 * 7 - 3;
```

However, comments are notorious for being incorrect or misleading. The above comment is a perfect example of an unnecessary comment. A better solution is to figure out a clearer way to express the code.



Replace comments with more expressive code.

One possible solution:

```
final int sessionLength = 16;
final int daysInWeek = 7;
final int daysFromFridayToMonday = 3;
int numberOfDays =
    sessionLength * daysInWeek - daysFromFridayToMonday;
```

Hmmm. Well, it's more expressive, but I'm not sure that `daysFromFridayToMonday` expresses exactly what's going on. This should demonstrate that there's not always a perfect solution. Refactoring is not an exact science. That shouldn't stop you from trying, however. Most changes *do* improve the code, and

someone (maybe you) can always come along after you and figure out an even better way. For now, it's your call.

Java supplies another form of comment known as a *multiline comment*. A multiline comment starts with the two characters `/*` and ends with the two characters `*/`. Everything from the beginning slash through to the ending slash is ignored by the compiler.

Note that while multiline comments can nest single line comments, multiline comments cannot nest other multiline comments.

As an example, the Java compiler allows the following:

```
int a = 1;
/* int b = 2;
// int c = 3;
*/
```

while the following code will not compile:

```
int a = 1;
/* int b = 2;
/* int c = 3; */
*/
```

You should prefer single-line comments for the few places that you need to annotate the code. The multiline comment form can then be used for rapidly “commenting out” (turning off the code so that it is not read by the compiler) large blocks of code.

Javadoc
Comments

Javadoc Comments

Another use for multiline comments is to provide formatted code documentation that can later be used for automated generation of nicely formatted API documentation. These comments are known as javadoc comments, since there is a javadoc tool that will read your source files, look for javadoc comments, and extract them along with other necessary information in order to build documentation web pages. Sun's documentation for the Java APIs themselves was produced using javadoc.

A javadoc comment is a multiline comment. The distinction is that a javadoc comment starts with `/**` instead of `/*`. To the javac compiler, there is no difference between the two, since both start with `/*` and end with `*/`. The javadoc tool understands the difference, however.

Javadoc comments appear directly before the Java element they are documenting. Javadoc comments can appear before fields, but most typically they are used to document classes and methods. There are rules for how a javadoc comment must be formatted in order to be parsed correctly by the javadoc compiler.

The primary reason for producing javadoc web pages is to document your code for external consumption by other project teams or for public distribution. While you can code javadoc comments for every Java element (fields, methods, classes, etc.), you should only expend the time to produce javadoc comments for things you want to expose. Javadoc comments are intended for telling a client developer how to work with a class.

In a team doing test-driven development, Javadoc comments are of lesser value. If done properly, the tests you produce using test-driven development provide far better documentation on the capabilities of a class. Practices such as pair programming and *collective code ownership*, where developers work on all parts of the system, also minimize the need for producing javadoc comments.

If you code concise, well-named methods, with well-named parameters, the amount of additional text that you should have to write in a Javadoc comment should be minimal. In the absence of text, the Javadoc compiler does a fine job of extracting and presenting the names you chose.

As a brief exercise, provide a Javadoc comment for the `CourseSession` class—the single-argument constructor—and for one of the methods within the class.

Here are the Javadoc comments I came up with:

```
package studentinfo;

import java.util.*;

/**
 * Provides a representation of a single-semester
 * session of a specific university course.
 * @author Administrator
 */
class CourseSession {
    private ArrayList<Student> students = new ArrayList<Student>();
    private Date startDate;

    CourseSession() {
    }

    /**
     * Constructs a CourseSession starting on a specific date
     *
     * @param startDate the date on which the CourseSession begins
     */
}
```

```

    */
    CourseSession(Date startDate) {
        this.startDate = startDate;
    }

    /**
     * @return Date the last date of the course session
     */
    Date getEndDate() {
        ...
    }

```

Take particular note of the @ keywords in the javadoc comments. When you look at the web pages produced by the Javadoc command, it will be apparent what the Javadoc compiler does with the tagged comments. The Javadoc keywords that you will want to use most frequently are @param, to describe a parameter, and @return, to describe what a method returns.

There are lots of rules and many additional @ keywords in Javadoc. Refer to the Javadoc documentation for further information. The Javadoc documentation can be found in the API documentation for the Java SDK (either online or downloaded), under the tool docs for your platform.

Once you've added these or similar comments to your code, go to the command line. (If you are using an IDE, you may be able to generate the documentation from within the IDE.) You will want to create a new empty directory in which to store the generated docs, so that you can easily delete it when you regenerate the Javadoc. Navigate to this empty directory¹⁰ and enter the following command:

```
javadoc -package -classpath c:\source;c:\junit3.8.1\junit.jar studentinfo
```

The javadoc program will produce a number of .html files and a stylesheet (.css) file. Open the file index.html in a web browser and take a few minutes to see what this simple command has produced (Figure 2.5). Fairly impressive, no?

Well, yes and no. I'm embarrassed by the comments I had you put in for the methods. They really added nothing that the code didn't already state. The @param keyword simply restates information that can be gleaned from the parameter type and name. The @return keyword restates information that can be derived from the method name and return type. If you find a need for @return and @param keywords, try to rename the parameters and the method to eliminate this need.

¹⁰ Instead of navigating to the empty directory, you can also redirect the output from the javadoc command using its -d switch.

Exercises

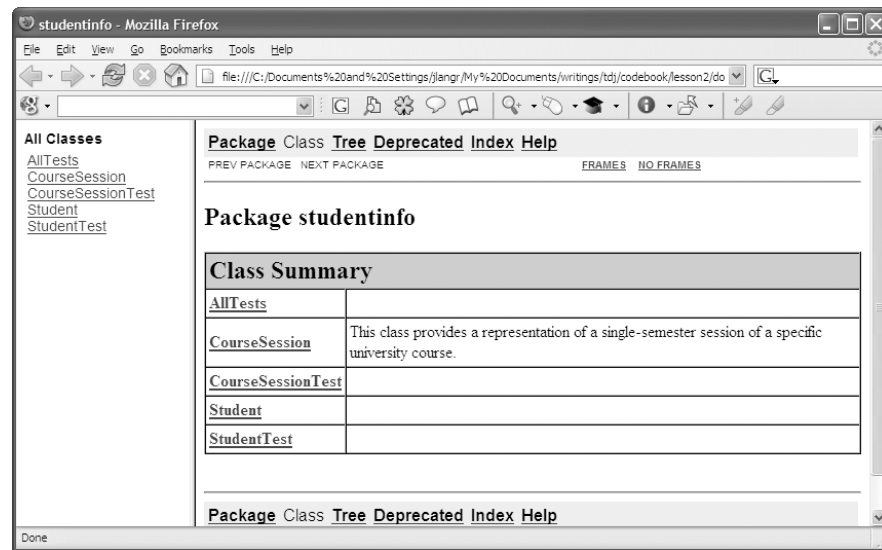


Figure 2.5 Your Own API Pages

Remove the comments for the constructor and the methods completely, but leave the comment for the class—it does provide a bit more value to the reader. Then rerun the javadoc command, bring up the web pages again, and see if you feel any useful information was lost. Your opinion may differ.

Exercises

1. Add a test to TestPawn that creates a pawn without a color. Why does this generate a compile error? (Hint: Think about default constructors.) Fix the compiler error by adding a second constructor that constructs a white pawn by default.
2. Make the constants for the two colors static and move them onto the Pawn class.
3. Pawns aren't very useful without a board. Use a test to define a Board class. Assert that the board starts with zero pieces on it. Follow the TDD sequence: Write the smallest test possible. Prove failure with a red bar or a compile error. Incrementally add small bits of code to obtain a clean compile or green bar.
4. Develop code to allow pawns to be added to the board. In a test, add a black and white pawn to the board. Each time you add a pawn, as-

sert that the piece count is correct. Also, each time you add a pawn, obtain a list of pieces from the board and ensure that it contains the expected pawn objects.

5. Write javadoc for each of the production classes and methods you have created so far. Be careful in doing so: Do not duplicate information that your methods already impart! The javadoc should be supplementary information.
6. Move the four tests and classes you have created into a package of their own. Name this package `chess`. Resolve compilation failures and get to a green bar again. Also, replace fully qualified class names for `List` and `ArrayList` by using an `import` statement.
7. Move `TestPawn` and `Pawn` into a package named `pieces` and resolve any problems discovered along the way.
8. Ensure that nothing other than a `Pawn` can be added to the board. Try adding a new `Integer("7")` to the list of pawns and see the resulting compiler error.
9. Create a test suite that runs each test class.
10. Look through the code you have written so far. Ensure there is no duplication anywhere in the code. Remember that the test code is code as well. Use the `setUp` method if appropriate.

